OS SKILL 3

History.c:

```c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <termios.h>

#define MAX 100

int main() {
    char history[10][MAX], buf[MAX];
    int count = 0, pos, len;
    char c, seq[2];

    struct termios old, raw;
    tcgetattr(0, &old);
    raw = old;
    raw.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(0, TCSANOW, &raw);

    while (1) {
        len = 0;
        pos = count;
        buf[0] = '\0';

        printf("cmd> ");
        fflush(stdout);

        while (read(0, &c, 1) == 1) {

            if (c == '\n' || c == '\r')
                break;

            /* Arrow key escape sequence */
            if (c == 27) {
                read(0, &seq[0], 1);
                read(0, &seq[1], 1);

                if (seq[0] == '[' && seq[1] == 'A' && pos > 0)
                    pos--;                          // UP arrow

                else if (seq[0] == '[' && seq[1] == 'B' && pos < count)
                    pos++;                          // DOWN arrow

                if (pos < count)
                    strcpy(buf, history[pos]);
                else
                    buf[0] = '\0';
```

```
                len = strlen(buf);

                printf("\r\033[2Kcmd> %s", buf);
                fflush(stdout);
            }

            else if (c == 127 && len > 0) {
                len--;
                buf[len] = '\0';
                printf("\b \b");
                fflush(stdout);
            }

            else if (len < MAX - 1) {
                buf[len++] = c;
                buf[len] = '\0';
                write(1, &c, 1);
            }
        }

        printf("\n");

        if (strcmp(buf, "exit") == 0)
            break;

        if (len > 0 && count < 10)
            strcpy(history[count++], buf);

        printf("Command entered: %s\n", buf);
    }

    tcsetattr(0, TCSANOW, &old);
    return 0;
}
```

```
cmd> ls
Command entered: ls
cmd> pwd
Command entered: pwd
cmd> echo hello
Command entered: echo hello
cmd> exit
```

Memory.c:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node {
    char *data;
    struct Node *next;
} Node;

char *readLine() {
    int size = 8, len = 0;
    char *buf = malloc(size);
    char c;

    while ((c = getchar()) != '\n') {

        if (len + 1 >= size) {
            size *= 2;

            char *temp = realloc(buf, size);

            if (temp == NULL) {
                free(buf);
                return NULL;
            }

            buf = temp;
        }

        buf[len++] = c;
    }

    buf[len] = '\0';
    return buf;
}

int main() {
    Node *head = NULL, *temp;

    for (int i = 0; i < 3; i++) {

        printf("Enter text: ");

        Node *newNode = malloc(sizeof(Node));

        newNode->data = readLine();
```

```
        newNode->data = readLine();
        newNode->next = head;
        head = newNode;
    }

    printf("\nStored Data:\n");

    temp = head;

    while (temp != NULL) {
        printf("%s\n", temp->data);
        temp = temp->next;
    }

    /* Release memory */
    while (head != NULL) {
        temp = head;
        head = head->next;

        free(temp->data);
        free(temp);
    }

    return 0;
}
```

```
nishanth@LAPTOP-6Q3ONO7H:~/skill3$ ./memory
Enter text: hello
Enter text: operating systems
Enter text: this is very long command for testing dynamic buffer resisizing

Stored Data:
this is very long command for testing dynamic buffer resisizing
operating systems
hello
```

```
nishanth@LAPTOP-6Q3ONO7H:~/skill3$ valgrind --leak-check=full ./memory
==1787== Memcheck, a memory error detector
==1787== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==1787== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==1787== Command: ./memory
==1787==
Enter text: hello
Enter text: operating systems
Enter text: this is a very long code for dynamic buffer resiszing

Stored Data:
this is a very long code for dynamic buffer resiszing
operating systems
hello
==1787==
==1787== HEAP SUMMARY:
==1787==     in use at exit: 0 bytes in 0 blocks
==1787==   total heap usage: 13 allocs, 13 frees, 2,280 bytes allocated
==1787==
==1787== All heap blocks were freed -- no leaks are possible
==1787==
==1787== For lists of detected and suppressed errors, rerun with: -s
==1787== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```